dipandang sebagai optimasi gradient descent dalam ruang fungsi, dan menjadi dasar bagi algoritma populer seperti XGBoost, LightGBM, dan CatBoost yang banyak digunakan dalam kompetisi data science.

```
F_m(x) = F_{m-1}(x) + learning_rate * h_m(x)
```

dengan h_m(x) dilatih untuk mengoreksi residual dari F_{m-1}(x), dan learning_rate mengontrol kontribusi tiap model baru (analog dengan konsep shrinkage).

Tabel 1. Perbandingan Random Forest vs Gradient Boosting

Aspek	Random Forest (Bagging)	Gradient Boosting
Cara melatih	Paralel, independen	Sekuensial, saling bergantung
Fokus utama	Mengurangi variance	Mengurangi bias (dan variance)
Sensitivitas terhadap outlier	Relatif robust	Lebih sensitif (fokus pada residual/kesalahan)
Kecepatan pelatihan	Dapat diparalelkan, relatif cepat	Sekuensial, umumnya lebih lambat
Kinerja umum	Sangat baik, konsisten	Seringkali sedikit lebih akurat jika di-tuning dengan baik

9.7. Kelebihan dan Kekurangan Ensemble Learning

Kelebihan	Kekurangan
Umumnya jauh lebih akurat dan stabil dibandingkan model tunggal	Kurang interpretable dibandingkan Decision Tree tunggal ("black box" ensemble)
Random Forest robust terhadap overfitting dibandingkan pohon keputusan tunggal	Boosting rentan overfitting bila jumlah estimator terlalu banyak tanpa regularisasi
Menyediakan feature importance yang berguna untuk interpretasi model	Boosting lebih sensitif terhadap noise dan outlier dibandingkan bagging

9.8. Kompleksitas

Random Forest berkompleksitas O(n_estimator · n · log n · m); karena tiap pohon dapat dilatih secara paralel, waktu pelatihan aktual dapat dipangkas signifikan. Gradient Boosting bersifat sekuensial sehingga tidak dapat diparalelkan pada level model, meskipun implementasi modern seperti XGBoost menerapkan berbagai optimasi.

9.9. Studi Kasus

Deteksi Klaim Asuransi Fraud

Aspek	Uraian
Masalah	Perusahaan asuransi memerlukan model prediksi klaim fraud yang akurat sekaligus dapat dijelaskan kepada tim investigasi.
Solusi & Proses	Random Forest dipilih karena menyeimbangkan akurasi dengan feature importance. Gradient Boosting (XGBoost) diuji dan sedikit lebih akurat, namun Random Forest dipertahankan demi kemudahan menjelaskan OOB error dan feature importance kepada tim non-teknis.
Hasil & Interpretasi	Akurasi jauh melampaui Decision Tree tunggal sebelumnya. Feature importance mengungkap "selisih waktu kejadian–pelaporan klaim" dan

NOTE 40 23mm f/1.8 1/50s ISO817

	"riwayat klaim sebelumnya" sebagai faktor paling berpengaruh wawasan yang langsung dapat ditindaklanjuti.

9.10. Contoh Perhitungan

Ilustrasi sederhana OOB: jika sebuah bootstrap sample berukuran n diambil dari dataset berukuran n (dengan pengembalian), probabilitas satu data tidak terpilih pada satu kali pengambilan adalah (1 - 1/n). Probabilitas data tersebut tidak terpilih pada seluruh n kali pengambilan (independen) adalah:

```
P(tidak terpilih) = (1 - 1/n)^n

Untuk n besar, nilai ini mendekati:
(1 - 1/n)^n -> e^(-1) = 0.3679 (~36.8%)
```

Artinya, sekitar 36.8% data tidak pernah terpilih pada satu bootstrap sample dan menjadi data "out-of-bag" untuk pohon tersebut inilah dasar teoritis mengapa OOB error dapat menjadi estimasi validasi yang valid.

9.11. Implementasi Python

Praktikum 9.1 Persiapan Library

```
1. import numpy as np
2. import pandas as pd
3. import matplotlib.pyplot as plt
4. from sklearn.datasets import load_wine
5. from sklearn.model_selection import train_test_split
6. from sklearn.ensemble import RandomForestClassifier,
7.     GradientBoostingClassifier,AdaBoostClassifier
8. from sklearn.tree import DecisionTreeClassifier
9. from sklearn.metrics import accuracy_score, f1_score
```

Praktikum 9.2 Import Dataset

```
1. data = load_wine(as_frame=True)
2. X, y = data.data, data.target
3. X_train,X_test,y_train,y_test = train_test_split(X,y,
4.     test_size=0.2,random_state=42,stratify=y)
```

Praktikum 9.3 Preprocessing dan Training Model

```
1. rf = RandomForestClassifier(n_estimators=300,
2.     oob_score=True,random_state=42)
3. rf.fit(X_train, y_train)
4.
5. ada = AdaBoostClassifier(n_estimators=100, random_state=42)
6. ada.fit(X_train, y_train)
7.
8. gb = GradientBoostingClassifier(n_estimators=100,
9.     learning_rate=0.1,random_state=42)
10. gb.fit(X_train, y_train)
11.
12. tree_tunggal = DecisionTreeClassifier(random_state=42).fit(
13.     X_train,
14.     y_train)
```

Praktikum 9.4 Evaluasi

```
1. print("OOB Score Random Forest:", round(rf.oob_score_, 3))
2.
```


NOTE 40 23mm f/1.8 1/50s ISO742

```
for nama,model in [("Decision Tree Tunggal",tree_tunggal),
    ("Random Forest",rf),("AdaBoost",ada),
    ("Gradient Boosting",gb)]:
    acc = accuracy_score(y_test, model.predict(X_test))
    print(f"{nama}: Akurasi Test = {acc:.3f}")
```

Praktikum 9.5 Visualisasi

```
importances = rf.feature_importances_
idx = np.argsort(importances)[::-1][:8]
plt.barh([data.feature_names[i] for i in idx][::-1],
    importances[idx][::-1])
plt.xlabel("Feature Importance")
plt.title("Feature Importance - Random Forest")
plt.show()
```

9.11.1. Interpretasi

Model ensemble (Random Forest, AdaBoost, Gradient Boosting) umumnya menunjukkan akurasi yang lebih tinggi dan lebih stabil dibandingkan Decision Tree tunggal. OOB Score pada Random Forest memberikan estimasi performa yang dapat dibandingkan langsung dengan akurasi test set, sebagai validasi tambahan bahwa model tidak overfitting secara signifikan.

9.12. Latihan Praktikum

- Praktikum 1: Pengaruh n_estimators

Tujuan: Mengamati pengaruh jumlah pohon (n_estimators) terhadap performa dan OOB score Random Forest.

Praktikum 9.6

```
for n in [10, 50, 100, 300, 500]:
    model = RandomForestClassifier(n_estimators=n,
        oob_score=True,random_state=42)
    model.fit(X_train, y_train)
    print(f"n_estimators={n}: OOB={model.oob_score_:.3f},
        Test Acc={accuracy_score(y_test,
        model.predict(X_test)):.3f}")
```

- Praktikum 2: Perbandingan Empat Model

Tujuan: Membandingkan Decision Tree, Random Forest, AdaBoost, dan Gradient Boosting secara menyeluruh menggunakan Stratified k-Fold CV.

Instruksi: Gunakan StratifiedKFold(n_splits=5) dan cross_val_score dengan scoring='f1_macro' untuk keempat model. Sajikan hasilnya dalam tabel dan tuliskan kesimpulan model mana yang terbaik untuk dataset yang Anda pilih.

NOTE 40
23mm f/1.8 1/50s ISO870